

Future Roadmap — Embedded C++ Learning Expansion

Platform: nRF52840 | **RTOS:** Zephyr | **Last Updated:** 2026-05-27

This document captures future topics, features, upgrades, and project ideas for the

Embedded C++ learning plan. Inputs gathered from DeepSeek AI + internal planning.

Table of Contents

- 1. [Advanced C++ & Embedded Topics](#)
 - 2. [New Hardware & Protocol Modules](#)
 - 3. [Project Ideas](#)
 - 4. [Modern C++20/23 Features](#)
 - 5. [Testing & CI/CD](#)
 - 6. [Real-World Industry Topics](#)
 - 7. [Zephyr-Specific Advanced Topics](#)
 - 8. [Performance & Optimization](#)
 - 9. [Planned Day Expansions \(Day 31–45\)](#)
 - 10. [Upgrade Ideas for Existing Days](#)
 - 11. [Tooling & Developer Experience](#)
 - 12. [Implementation Priority Matrix](#)
-

1. Advanced C++ & Embedded Topics

Topics not covered in the 30-day plan that professional embedded engineers need.

1.1 Advanced DMA & Multi-channel Transfers

- nRF52840's **PPI (Programmable Peripheral Interconnect)** — hardware task/event chaining without CPU involvement
- DMA double-buffering patterns for zero-copy ADC/I2S streaming
- `nrfx_dma` API, descriptor chaining, circular DMA for audio
- **Why it matters:** Eliminates CPU involvement in high-speed data transfers — essential for audio, high-rate ADC, and USB

1.2 RTOS Scheduler Internals & Custom Scheduling

- Zephyr scheduler internals: run queue, cooperative vs preemptive, timeslicing
- `CONFIG_SCHED_DEADLINE` — deadline scheduling for audio and control systems
- Preemption thresholds (`k_thread_priority_set()` , `CONFIG_PREEMPT_ENABLED`)
- Understanding context switch cost and how to minimize it
- **Why it matters:** Knowing the scheduler internals helps you tune thread priorities and avoid priority inversion bugs

1.3 Fault Recovery & Watchdog Strategies

- **nRF Watchdog (WDT)** peripheral — hardware watchdog that resets the chip if not fed in time
- Dual-stage watchdog: HW watchdog + software task monitors
- Multi-level fault handlers: HardFault, MemManage, BusFault with stack trace reconstruction
- Zephyr `CONFIG_FAULT_DUMP=2` — full register + stack dump on crash
- Non-volatile crash log: save fault info to Flash before reset using `.noinit` section
- **Why it matters:** Production firmware must handle hardware faults gracefully and recover automatically

1.4 Bootloader Design & Secure DFU

- **MCUboot** integration with Zephyr — dual-image (A/B) secure boot
- Signed firmware updates (ed25519 / RSA-2048)
- `west flash --recover` and UICR (User Information Configuration Registers)
- `mcumgr` protocol for OTA updates over BLE, UART, or USB
- `CONFIG_BOOTLOADER_MCUBOOT=y` + image signing with `imgtool`
- **Why it matters:** Every commercial embedded product needs a safe, secure OTA update mechanism

1.5 Real-Time Debugging with Trace

- **SEGGER SystemView** — real-time RTOS-aware trace (tasks, ISRs, kernel events)
- ETM (Embedded Trace Macrocell) on Cortex-M4 via J-Trace
- Custom trace events: `SEGGER_SYSVIEW_Print()`, `SEGGER_SYSVIEW_OnUserStart()`
- `CONFIG_SEGGER_SYSTEMVIEW=y` in Zephyr
- Trace-driven profiling — find actual bottlenecks, not guessed ones
- **Why it matters:** RTOS-aware tracing is the only reliable way to debug timing and scheduling issues

1.6 Advanced Cryptography & Secure Elements

- nRF52840 **CC310 hardware accelerator** — AES-128/256, SHA-256, HMAC, ECDH, ECDSA
- **KMU (Key Management Unit)** — hardware key storage, key push to CC310
- `nrf_cc310_bl` library for bootloader crypto
- TLS 1.3 over BLE using Mbed TLS with CC310 backend
- **Why it matters:** IoT security is mandatory — hardware crypto is 10–100x faster than software

1.7 FAT Filesystem & Wear Leveling

- **LittleFS** — power-loss safe filesystem designed for Flash
- Zephyr FS API: `fs_open()`, `fs_read()`, `fs_write()`, `fs_mount()`
- Flash wear leveling: write endurance on nRF52840 internal Flash (~10,000 cycles)
- External QSPI Flash (MX25R6435F on nRF52840DK) for larger storage
- `CONFIG_FILE_SYSTEM_LITTLEFS=y`, `CONFIG_FLASH_MAP=y`

- **Why it matters:** Data logging, configuration storage, OTA staging all require reliable filesystem access

1.8 Advanced Sensor Fusion & Signal Processing

- **CMSIS-DSP** library (`arm_math.h`) — optimized DSP for Cortex-M4
- Kalman filter implementation for IMU fusion (accelerometer + gyroscope)
- Fixed-point arithmetic (`q15_t` , `q31_t`) vs floating point trade-offs
- FFT via `arm_rfft_fast_f32()` for audio/vibration analysis
- Moving average, median filter, IIR/FIR digital filters
- **Why it matters:** Sensor fusion and signal processing are core skills for any sensing application

1.9 Network Stack Internals

- Zephyr's `NET_BUF` pool management — zero-copy network buffers
- LwIP integration for TCP/IP over Ethernet or Wi-Fi
- CoAP and MQTT-SN for constrained IoT devices
- `CONFIG_NET_BUF_USER_DATA_SIZE` , custom net buffer pools
- **Why it matters:** Networking is increasingly common even in deeply embedded devices

1.10 Advanced Interrupt Nesting & Latency Analysis

- ARM NVIC priority grouping — preemption vs subpriority bits
- ISR-safe double-buffering with memory barriers (`__DMB()` , `__DSB()`)
- `IRQ_CONNECT()` with compile-time priority assignment
- Measuring interrupt latency with GPIO toggle + oscilloscope
- Interrupt storm prevention and rate-limiting
- **Why it matters:** Missed interrupts and excessive latency cause data loss and timing errors in production

1.11 Multi-Core & Asymmetric Multiprocessing (nRF5340)

- **nRF5340** — dual Cortex-M33 (application core + network core)
- Zephyr IPC service: `ipc_service_send()` , shared memory, RPMsg

- Splitting BLE stack to network core, application on app core
- **Why this matters:** The nRF5340 is the natural upgrade path from nRF52840 for production BLE products

1.12 Real-Time Clock & Calendar

- **nRF52840 RTC peripheral** — 32kHz crystal, 24-bit counter
- Zephyr RTC API ()
- Calendar functions, alarm generation, epoch time synchronization via BLE/NTP
- Low-power periodic wake from deep sleep using RTC alarm
- **Why it matters:** Timestamping sensor data and scheduling tasks by wall-clock time are common requirements

2. New Hardware & Protocol Modules

New peripheral/protocol modules to add as separate day topics or mini-projects.

#	Module	Peripheral / IC	Zephyr API	Priority
1	Bluetooth LE 5.x	nRF52840 radio	<code>bt_le_adv_start()</code> , <code>bt_gatt_notify()</code>	● High
2	USB Device (CDC ACM, HID)	nRF52840 USBD	<code>usb_enable()</code> , HID report	● High
3	CAN Bus (CANopen)	External MCP2515/ MCP2517FD via SPI	<code>can_send()</code> , <code>can_attach_isr()</code>	● Medium
4	I2S Audio & PDM Microphone	nRF52840 I2S, MP34DT05 MEMS mic	<code>i2s_trigger()</code> , PDM driver	● Medium

#	Module	Peripheral / IC	Zephyr API	Priority
5	NFC Type 4 Tag	nRF52840 NFCT + NT3H1101	<code>nfc_t4t_setup()</code> , NDEF	 Medium
6	6LoWPAN / OpenThread	nRF52840 802.15.4 radio	<code>otInstance</code> , OpenThread API	 Medium
7	QSPI NOR Flash (XIP)	nRF52840 QSPI, MX25R6435F	<code>nrfx_qspi</code> , <code>CONFIG_XIP</code>	 Medium
8	BME280 Environmental Sensor	I2C/SPI	<code>sensor_channel_get()</code> , Zephyr sensor API	 High
9	IMU — BNO055 / ICM-20948	I2C/SPI	Zephyr sensor API, quaternion	 Medium
10	GNSS — u-blox MAX-M10S	UART	NMEA parsing, <code>CONFIG_GPS</code>	 Low
11	PWM Motor Control	nRF52840 PWM peripheral	<code>pwm_pin_set_usec()</code> , BLDC	 Medium
12	Low-Energy Accelerometer	LIS2DH12 via SPI	Tap detection, motion wake	 Medium
13	E-Paper Display	SPI (IL0373, UC8151)	Custom SPI driver, framebuffer	 Low
14	Zigbee	nRF52840 + ZBOSS stack	Zigbee Coordinator/Router/End Device	 Low

3. Project Ideas

Complete embedded projects a learner can build after Day 30.

Project 1: BLE Sensor Tag with Cloud Dashboard 📡 High Value

Description: nRF52840 reads BME280 (temp, humidity, pressure) + battery voltage, advertises via BLE. A Raspberry Pi gateway receives data and pushes to MQTT broker → Grafana dashboard.

C++ concepts used: RAII driver classes, Singleton BLE manager, Ring buffer for local logging, LittleFS for data persistence, State machine for connection/advertising FSM

New topics covered: BLE Peripheral role, GATT services/characteristics, MQTT, LittleFS, ADC SAADC

nRF52840 (Sensor Tag) → BLE → RPi Gateway → MQTT → Grafana

Project 2: USB-CAN Bus Analyzer 🟡 Medium Value

Description: nRF52840 bridges CAN bus (via MCP2517FD over SPI) to USB host as a CDC ACM serial device. Features: CAN ID filtering, message timestamping, PCAP export for Wireshark.

New topics covered: USB Device stack, CAN FD driver, SPI master, DBC file parsing

Project 3: Low-Power Environmental Data Logger 📡 High Value

Description: 3-year battery life target. Deep sleep (System OFF) with RTC wake every 15 minutes, logs BME280 + light sensor to QSPI Flash, BLE connection for data download.

Target: < 5 μ A average current. Validated with Nordic PPK2.

New topics covered: System OFF mode, RTC alarm wake, QSPI Flash, power budget calculation

Project 4: Thread Mesh Smart Lighting Controller 🟡 Medium Value

Description: OpenThread mesh network of nRF52840 nodes. Each node controls PWM LED (dimnable, color). Central coordinator schedules scenes. OTA DFU via Thread multicast.

New topics covered: OpenThread, UDP multicast, MCUboot OTA, PWM color mixing

Project 5: Secure NFC Access Control System 🟡 Medium Value

Description: NFC tag emulation for key card authentication. HMAC-SHA256 challenge-response using nRF52840 CC310. Door lock via GPIO relay. Audit log in LittleFS with signed entries.

New topics covered: NFC Type 4 Tag emulation, CC310 crypto, LittleFS, signed log entries

Project 6: Real-Time Audio Spectrum Analyzer ● Low (Complex)

Description: I2S MEMS microphone input → FFT via CMSIS-DSP (`arm_rfft_fast_f32`) → 16-band display on SPI LED matrix → BLE audio level streaming.

New topics covered: I2S driver, PDM mic, CMSIS-DSP FFT, LED matrix SPI driver

Project 7: Automotive CAN Intrusion Detection System ● Low (Advanced)

Description: Monitors CAN bus traffic, detects anomalous message frequencies (potential replay attacks), logs incidents to Flash, sends alerts over BLE. Signed firmware via MCUboot.

New topics covered: CAN FD, statistical anomaly detection, MCUboot, BLE alerts

4. Modern C++20/23 Features

C++ language features not yet covered that are practical for embedded.

4.1 `std::span` (C++20) — Bounded Buffer Views

```
// Replace: void process(uint8_t *buf, size_t len)
// With:
void process(std::span<uint8_t> buf) {
    for (auto b : buf) { /* ... */ }
}
// Zero overhead – just a pointer + size struct
```

Use case: DMA buffer views, UART RX buffers, SPI transfer descriptors

4.2 `std::bit_cast` (C++20) — Safe Type Punning

```
// Replace unsafe union type punning:
float f = 3.14f;
uint32_t raw = std::bit_cast<uint32_t>(f); // Safe, well-defined
```

Use case: Serializing floats over UART, inspecting IEEE754 float representation

4.3 `std::expected` (C++23) — Error Handling Without Exceptions

```
std::expected<uint16_t, int> read_adc(uint8_t channel) {
    if (channel > 7) return std::unexpected(-EINVAL);
    return 1024U;
}

auto result = read_adc(3);
if (result) { use(*result); }
else      { LOG_ERR("Error: %d", result.error()); }
```

Use case: Replaces `Result` template from Day 22 with a standard type

4.4 `std::variant` (C++17) — Type-Safe Union

```
using Packet = std::variant<SensorPacket, CommandPacket, AckPacket>;
Packet pkt = decode_uart_frame(raw_bytes);
std::visit([](auto &p) { handle(p); }, pkt); // Type-safe dispatch
```

Use case: Protocol decoders, command parsers, CAN message demuxing

4.5 `std::source_location` (C++20) — Better Debug Logging

```
void log_error(int code, std::source_location loc = std::source_location::current()) {
    LOG_ERR("[%s:%d] Error: %d", loc.file_name(), loc.line(), code);
}

// Replaces: LOG_ERR("[%s:%d]", __FILE__, __LINE__, code);
```

4.6 `std::atomic_ref` (C++20) — Non-Owning Atomic Access

```
// ISR-shared flag without volatile:
bool data_ready = false;
std::atomic_ref<bool> atomic_flag(data_ready);
atomic_flag.store(true, std::memory_order_release); // ISR-safe
```

4.7 Coroutines (C++20) — Async Embedded Patterns

```
// Cooperative multitasking without RTOS threads:
Generator<uint16_t> adc_samples() {
    while (true) {
        co_yield read_adc_raw();
        k_msleep(10);
    }
}
```

Note: Coroutines require heap or pre-allocated frames — evaluate carefully for embedded

5. Testing & CI/CD

Building professional-grade test infrastructure for embedded projects.

5.1 Host-Based Unit Testing with Google Test + Ztest

- Build application logic natively on x86 with `CMake -DBOARD=native_posix`
- Use `gmock` to mock hardware HAL (GPIO, UART, SPI)
- Zephyr's built-in `ztest` framework for on-target tests
- Test state machines, protocol parsers, and business logic without hardware

```
west build -b native_posix -t run # Run tests on host
west twister -p native_posix -T tests/ # Run all tests
```

5.2 Hardware-in-the-Loop (HIL) Testing

- nRF52840DK + Raspberry Pi as automated test harness
- GPIO injection (simulate button presses, sensor signals)
- UART command verification (send command, check response)
- Power measurement automation with PPK2 REST API
- Pytest framework driving J-Link + PySerial

5.3 CI Pipeline with GitHub Actions

```
# .github/workflows/build.yml
strategy:
  matrix:
    board: [nrf52840dk_nrf52840, native_posix]
    config: [debug, release]
steps:
  - uses: zephyrproject-rtos/action-zephyr-setup@v1
  - run: west build -b ${matrix.board} --cmake-only
  - run: cppcheck --std=c++17 src/
  - run: clang-tidy src/**/*.cpp
```

- Artifact storage for firmware binaries (`.hex` , `.elf`)
- Binary size tracking — alert when binary grows > threshold
- Flash/RAM usage reporting via `west build --cmake-only` size output

5.4 Fuzz Testing for Protocol Handlers

- Use `libFuzzer` on host to test UART command parser, BLE GATT handlers
- AddressSanitizer (ASan) + UBSan for memory safety on host builds
- Corpus-based fuzzing for CAN message decoders

```
clang++ -fsanitize=fuzzer,address -o fuzz_uart fuzz_uart.cpp
./fuzz_uart corpus/
```

5.5 Regression Testing with Zephyr Twister

```
west twister -p nrf52840dk_nrf52840 --device-testing -T tests/  
west twister --coverage -p native_posix -T tests/
```

- Automated flash + run + collect results over J-Link
- Compare power consumption across commits (PPK2 integration)
- Binary size regression tracking

5.6 Static Analysis Integration

- `clang-tidy` with `cppcoreguidelines-` and `cert-` checks
 - `cppcheck` for undefined behavior and null pointer analysis
 - `pvs-studio` or `coverity` for commercial-grade analysis
 - Integrate into `west build` via CMake `CMAKE_C_CLANG_TIDY`
-

6. Real-World Industry Topics

What professional embedded engineers need beyond the technical basics.

6.1 MISRA-C++:2023 Compliance

What it is: A coding standard for safety-critical C++ (automotive, medical, aerospace).

Key rules for embedded:

- No dynamic memory allocation after init (`Rule 18-5-1`)
- No exceptions (`Rule 15-*`)
- No recursion (`Rule 7-5-4`)
- All code paths must be reachable (`Rule 0-1-1`)
- Use `clang-tidy --checks=misra-c++*` for automated checking

6.2 IEC 61508 / ISO 26262 Functional Safety

What it is: International standards for safety-critical systems.

- SIL (Safety Integrity Level) / ASIL (Automotive SIL) requirements
- Hardware fault tolerance: redundant sensors, voted outputs
- Software systematic capability — code review, testing evidence
- Tool qualification (compiler, RTOS must be qualified for the target SIL)
- **Zephyr SafeRTOS** as a certified RTOS alternative

6.3 Secure Boot & Trusted Execution Environment

- ARM TrustZone for Cortex-M (available on nRF5340)
- Trusted Firmware-M (TF-M) — secure partition manager
- Measured boot: hash each firmware component, verify chain of trust
- `CONFIG_TRUSTED_EXECUTION_SECURE=y` for nRF5340 secure core

6.4 OTA Update Reliability Patterns

- A/B (dual-bank) update scheme — always have a working firmware to fall back to
- Power-loss safe update: atomic swap after verification
- Version rollback protection (downgrade attacks)
- Signed delta updates — send only the diff, not the full image
- `CONFIG_BOOTLOADER_MCUBOOT=y` + `imgtool sign`

6.5 Regulatory Compliance Testing

- **FCC/CE EMC** pre-compliance testing (conducted + radiated emissions)
- **Bluetooth SIG qualification** — RF-PHY, GAP, GATT profile qualification
- **USB-IF certification** for USB products
- **UL 60730** for appliance control systems
- **RoHS / REACH** material compliance documentation

6.6 Embedded Code Review Practices

- Pair review for safety-critical paths (two-person rule)
 - Review checklist: overflow checks, ISR safety, error handling, stack usage
 - Architectural review: HAL boundaries, interface contracts
 - Using `diff` reviews in GitHub with inline comments on hardware-specific decisions
-

7. Zephyr-Specific Advanced Topics

Going deeper into Zephyr's ecosystem beyond the 30-day coverage.

7.1 Advanced Devicetree — Custom Bindings

```
# my-sensor.yaml (custom DTS binding)
description: My Custom Sensor
compatible: "my,sensor"
properties:
  reg:
    required: true
  spi-max-frequency:
    required: true
  int-gpios:
    type: phandle-array
    description: Interrupt GPIO pin
```

- `DT_FOREACH_STATUS_OKAY()` for iterating all compatible nodes
- `gpio-hog` nodes for automatic pin configuration at boot
- `fixed-partitions` for flash memory layout (MCUboot + app + storage)

7.2 Custom Zephyr Driver Development

Full driver from scratch for a new sensor:

1. Write `my_sensor.yaml` DTS binding
2. Implement `sensor_driver_api` struct
3. Handle device PM with `pm_device_action_cb_t`
4. Support multiple instances via `DT_FOREACH_STATUS_OKAY`
5. Add to Zephyr build with `CMakeLists.txt` + `Kconfig`

7.3 West Extension Commands

```
# west_commands/my_cmd.py
class SignFirmware(WestCommand):
    def do_run(self, args, unknown_args):
        subprocess.run(['imgtool', 'sign', ...])
```

- Custom `west sign`, `west provision`, `west test` commands
- Board provisioning scripts (write device certificate, keys)
- Automated CI/CD hooks for firmware signing and deployment

7.4 Zephyr Shell — Custom Commands

```
SHELL_CMD_REGISTER(sensor, NULL, "Sensor commands", NULL);
SHELL_CMD_REGISTER(sensor_read, &sensor, "Read sensor value", cmd_sensor_read);
// Access via: uart:~$ sensor read
```

- Interactive debugging via Zephyr shell over UART/RTT
- Tab completion, command history, help system
- `CONFIG_SHELL=y`, `CONFIG_SHELL_BACKEND_SERIAL=y`

7.5 Zephyr Logging — Advanced Usage

- Multiple log backends simultaneously (RTT + UART + Flash)
- `LOG_PANIC()` — flush all buffered log messages before crash
- Deferred logging — buffer in RAM, flush asynchronously
- Structured logging with log dictionaries for binary-efficient logging

- `CONFIG_LOG_MODE_DEFERRED=y` , `CONFIG_LOG_BACKEND_FLASH=y`

7.6 Power Management Framework — Advanced

- `pm_device_runtime_enable()` — automatic power gating when device is idle
- `sys_pm_constraint_set()` — prevent certain sleep states during critical operations
- Custom sleep states via DTS `power-states` node for nRF52840 specific modes
- `pm_notifier` hooks for pre/post sleep operations (flush logs, save state)

8. Performance & Optimization

Squeezing maximum performance from nRF52840 / Cortex-M4.

8.1 Cache-Aware Programming

- nRF52840 has **instruction cache** (64-byte lines) but **no data cache**
- Hot functions: align to cache line with `__attribute__((aligned(64)))`
- Place time-critical ISRs in RAM (`__attribute__((section(".ramfunc")))`) — avoids Flash wait states
- Measure impact with DWT cycle counter before/after optimization

8.2 Link-Time Optimization (LTO)

```
# CMakeLists.txt
set_property(TARGET app PROPERTY INTERPROCEDURAL_OPTIMIZATION TRUE)
```

```
CONFIG_LTO=y
```

- Cross-module inlining — small functions from other `.cpp` files get inlined

- Dead code elimination across the entire binary
- **Caution:** Can increase build time significantly; debug symbols may be affected

8.3 Flash Wait States & Instruction Prefetch

- nRF52840 Flash: 1 wait state at 64 MHz (default)
- `CONFIG_ICACHE=y` — enable nRF52840 instruction cache (reduces effective wait states)
- For loops running > 1000 iterations: copy to RAM with `__ramfunc` for zero-wait execution
- Use `CONFIG_CODE_DATA_RELOCATION` to place hot code in RAM

8.4 Zero-Copy DMA Patterns

- Chain DMA descriptors for continuous ADC sampling without CPU re-arming
- Use **PPI (Programmable Peripheral Interconnect)** to trigger DMA from timer compare events
- Double-buffered DMA: while CPU processes buffer A, DMA fills buffer B
- `nrfx_saadc` with `NRF_SAADC_BURST_ENABLED` for oversampling
- **Achieve:** 1 Msps ADC with zero CPU involvement between samples

8.5 FPU Usage & Fixed-Point Trade-offs

	Float (FPU)	Fixed-point (q15/q31)
Speed	~3 cycles (FPU hw)	~1-2 cycles
Power	Higher (FPU active)	Lower
Range	Dynamic	Fixed — careful scaling needed
Ease	Easy	Requires scaling math

- `CONFIG_FPU=y` — enable Cortex-M4F hardware FPU
- CMSIS-DSP `q15_t` / `q31_t` types for power-efficient sensor fusion
- Use float for prototyping, convert to fixed-point for production

8.6 Interrupt Latency Optimization

- Measure IRQ latency: GPIO toggle at IRQ entry → logic analyzer → time from event to first instruction
- Minimize critical sections (`irq_lock()` duration < 1 μs target)
- Use `ISR_DIRECT_DECLARE` for ultra-low latency ISRs (bypasses Zephyr RTOS overhead)
- **nRF52840 measured IRQ latency:** ~2 μs from GPIO edge to first ISR instruction (with Zephyr)

8.7 Stack Size Right-Sizing

```
# After CONFIG_THREAD_STACK_INFO=y and running:
uart:~$ kernel stacks
# Output: thread name, stack size, current usage, peak usage
# Right-size: peak_usage + 128 bytes safety margin
```

- Over-allocated stacks waste precious RAM (256KB total on nRF52840)
- Under-allocated stacks cause hard-to-reproduce stack overflow bugs
- Use `CONFIG_STACK_SENTINEL=y` + `CONFIG_STACK_USAGE=y` during testing

9. Planned Day Expansions (Day 31–45)

New days to add after completing the 30-day plan.

Day	Topic	Prerequisite Days
31	Bluetooth LE — Advertising & GAP	Day 27, 28
32	Bluetooth LE — GATT Services & Characteristics	Day 31
33	Bluetooth LE — Pairing, Bonding & Security	Day 32, 22
34	USB Device Stack — CDC ACM UART	Day 14, 27

Day	Topic	Prerequisite Days
35	MCUboot & Secure OTA Updates	Day 9, 22
36	LittleFS — Filesystem on Flash	Day 9, 12
37	CMSIS-DSP — Signal Processing on Cortex-M4	Day 8, 18
38	BME280 Driver — Full Zephyr Sensor Driver	Day 21, 14
39	Zephyr Shell — Custom Command Interface	Day 30, 7
40	DMA Deep Dive — PPI & Zero-copy Patterns	Day 8, 13
41	SEGGER SystemView — RTOS-Aware Tracing	Day 7, 27
42	Watchdog & Fault Recovery	Day 8, 22
43	Unit Testing — Ztest + Native POSIX	Day 22, 30
44	CI/CD Pipeline for Embedded	Day 43
45	OpenThread — 6LoWPAN Mesh Network	Day 31, 27

10. Upgrade Ideas for Existing Days

Improvements to the current 30 days based on learner feedback.

Day	Current State	Proposed Upgrade
Day 1	Basic setup	Add QEMU simulator target for learners without hardware
Day 2	LED blink	Add PWM breathing LED using <code>pwm_pin_set_usec()</code>
Day 7	Basic logging	Add SEGGER RTT backend, show SystemView integration
Day 8	ARM arch overview	Add hands-on: read nRF52840 FICR (chip ID, flash size) registers

Day	Current State	Proposed Upgrade
Day 13	ISR basics	Add nRF52840 GPIOTE Tasks (hardware pin toggle without CPU)
Day 14	GPIO + UART	Add DMA-driven UART TX with <code>uart_tx()</code> async API
Day 21	Driver architecture	Add real BME280 or SHT31 sensor driver as example
Day 25	State machines	Add hierarchical state machine using Zephyr SMF parent states
Day 26	Ring buffers	Add <code>net_buf</code> pool example (used in BLE/networking)
Day 29	Power management	Add PPK2 integration, show actual current waveform
Day 30	Capstone	Add BLE characteristic to mirror UART shell commands

11. Tooling & Developer Experience

Tools and workflow improvements for a better embedded development experience.

11.1 VS Code + nRF Connect Extension

- **nRF Connect for VS Code** — official Nordic IDE extension
- IntelliSense for Zephyr headers, DTS files, Kconfig
- Integrated build, flash, debug (no terminal needed)
- DeviceTree viewer for visual DTS inspection
- [Install guide](#)

11.2 nRF Power Profiler (Desktop App)

- Real-time current measurement with Nordic PPK2
- Correlate current spikes with code events via GPIO timestamp markers

- Export CSV for post-processing in Python/pandas

11.3 Wireshark + nRF Sniffer

- **nRF Sniffer for Bluetooth LE** — captures BLE packets on air
- Plugin: `nRF Sniffer for 802.15.4` for Thread/Zigbee
- Live packet decode in Wireshark — invaluable for protocol debugging

11.4 Ozone (SEGGER Debugger)

- Source-level debugger with RTOS-awareness
- Timeline view, memory sampling, live variable watch
- Better than GDB CLI for complex multi-threaded debugging

11.5 clang + compile_commands.json

- `west build -- -DCMAKE_EXPORT_COMPILE_COMMANDS=ON`
- Enables proper IntelliSense for Zephyr's complex include tree
- Integrates with VS Code `clangd` extension for accurate completions/error highlighting

12. Implementation Priority Matrix

Rank future items by learning value vs implementation effort.

Item	Value	Effort	Priority
BLE LE (Days 31-33)	★★★★★★	🔨🔨🔨	🔴 Do first

Item	Value	Effort	Priority
BME280 Driver (Day 38)	★★★★★	🔨🔨	🔴 Do first
MCUboot OTA (Day 35)	★★★★★★	🔨🔨🔨	🔴 Do first
Ztest + CI/CD (Day 43-44)	★★★★★	🔨🔨🔨	🔴 Do first
LittleFS (Day 36)	★★★★★	🔨🔨	🟡 Do next
CMSIS-DSP (Day 37)	★★★★	🔨🔨	🟡 Do next
USB Device (Day 34)	★★★★	🔨🔨🔨	🟡 Do next
SystemView Tracing (Day 41)	★★★★	🔨	🟡 Do next
Watchdog/Fault (Day 42)	★★★★★	🔨	🟡 Do next
CAN Bus	★★★	🔨🔨🔨	🟢 Later
OpenThread (Day 45)	★★★★	🔨🔨🔨🔨	🟢 Later
Audio/I2S	★★★	🔨🔨🔨🔨	🟢 Later
MISRA Compliance	★★★★	🔨🔨🔨🔨	🟢 Later

DeepSeek AI Review Notes

Review Date: 2026-05-27

Model: deepseek-chat

DeepSeek contributed to sections 1-8 of this document. Key additional insights:

>

- Highlighted **PPI (Programmable Peripheral Interconnect)** as a uniquely nRF feature often overlooked — hardware event chaining with zero CPU involvement

- Recommended **SEGGER SystemView** as the single highest-ROI debugging tool for RTOS-based firmware

- Flagged **MISRA-C++:2023** as increasingly required even for non-safety-critical products due to customer/insurance requirements

- Noted `std::span` (C++20) as the most immediately practical modern C++ feature for embedded — already supported by GCC 10+ which Zephyr uses

- Suggested **nRF5340 multi-core** as the natural upgrade path after nRF52840 mastery

- Emphasized **Hardware-in-the-Loop (HIL) testing** as the gap between hobby and professional embedded development

Document is living — update as new topics are completed or new ideas arise.

Each completed item should be checked off and linked to its implementation day.